

Index, index on the wall who is the brightest of them all?



Introduction

Despite the popularity and fast growth of the ORACLE RDBMS installed base and the resulting publication of papers, presentations and studies, surprisingly few have been about indexes. The data warehouse cult which gave rise to the bitmap index and the subsequent release of ORACLE8 with its whole new arsenal of indexes such as partitioned indexes, reverse key indexes, the index organized table added to the confusion about this topic. This paper will attempt to shed some new light on the concept of index structures and their use in the ORACLE RDBMS. The desired outcome is a better understanding - and therefore a better use - of the various types of indexes.

The mother of all indexes:

The B* tree data structure

An index is a data structure that belongs to a table or a cluster. The goal of an index is similar to the index of a docu-

ment or a book, i.e. to be able to locate the desired data faster than scanning the underlying structure top to bottom. Given a certain data value, the index points directly to the location of the data (see figure 1).

The data structure used for all indexes in ORACLE, including the bitmap index, is the B*-tree. Figure 1 depicts an overview of a B*-tree. The 'leaves' (the bottom blocks) maintain the 'keys' of the index. In ORACLE's implementation these leaf blocks contain the column values which were indexed as well as the ROWIDs pointing directly to the actual data. The key values in the leaf blocks are ordered in key values - ROWID order to facilitate range scans. The leaves are doubly linked together thus providing a sequential access path for traversing the keys. The 'branches' of the tree are the intermediate blocks that direct a traversal to the correct leaf block. Each level of the tree is an index to the next level, and the

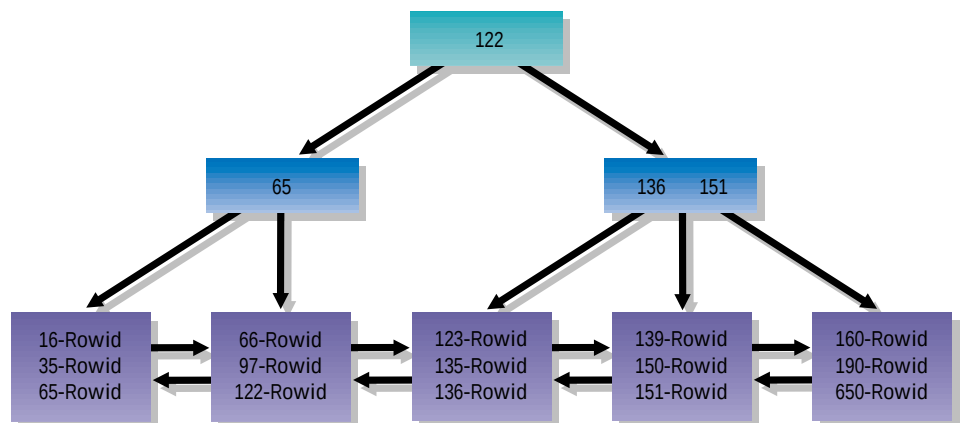


Figure 1: The B*-tree

By Walter Schenk
Managing Principal Consultant,
SoluTech Consulting Services

*As presented at the 1998
International Oracle User Group*

lowest level (the leaf level) is an index to the data itself. In ORACLE, the branch blocks are compressed for space efficiency. The 'height' of a B*-tree is the number of levels that make up the tree from the root (the ultimate parent) to the leaves.

The B*-tree data structure has a number of characteristics that are extremely attractive for data retrieval. First it provides rapid random access to data. Second, it also provides an excellent way to scan a range of key values because the leaf blocks are ordered and linked together. Third, the B*-tree is always balanced, i.e. all the leaf nodes are at the same level of the tree which means that the time required to retrieve an index key is always almost identical no matter what the key value is. Fourth, inserts, updates, and deletes are very efficient while at the same time maintaining key - ROWID order for fast retrieval. Fifth, the performance is very good for both small as well as large tables and doesn't degrade much as the table grows. This last characteristic is due to the fact that the height of the index tree doesn't change much.

The usefulness of an index is extremely difficult to predict. This obviously explains the vagueness of the rule of thumb in the Oracle manuals. In summary, there are three main factors which contribute to the usefulness of an index.

The first factor is the data distribution. Clustered data - i.e. the closer together the desired data is placed - makes indexes perform better than uniformly or randomly distributed data. The second factor is the effectiveness of a full table scan. The DBA can partially control this factor by setting a more appropriate value for the ORACLE init.ora parameter `db_file_multiblock_read_count`. It is also dependent on the underlying operating system because the effectiveness of a full table scan depends on how fast the system can retrieve contiguous data blocks from disk. A third factor is the cache hit ratio for the selected data. If all data is already in cache prior to the execution of the query, then all incurred I/Os will be logical I/Os. If no data is in cache prior to the execution of the query, then a large number of physical I/Os will incur in addition to logical I/Os. In most situations some data will be in cache and both LIOs and PIOs will take place. Therefore knowledge of the circumstances under which a query is run may greatly assist in determining whether or not an index would be useful.

The size of a B* tree index

The theoretical maximum number of entries in a B* tree index that is fully, densely populated is given by the following formula:

$$\text{Index Entries} = \text{BranchMax} (\text{Height} - 1) * \text{LeafMax}$$

Where:

Index Entries: the total theoretical maximum number of index entries in the leaf blocks of an index

BranchMax: The maximum number of entries in a single index branch block when fully, densely populated

LeafMax: The maximum number of entries in a single index leaf block when fully, densely populated

Height: The total number of 'levels' in an index structure; level includes the 'root' block of the index as well as the leaf blocks. Sometimes this is also called the 'depth' of an index.

Below are two examples of this 'theoretical' maximum table size for an index of a certain height (3 and 4). It should be recognized that this is a theoretical maximum, i.e. all the index blocks (leaf and branch) are fully populated. Of course, a more realistic picture unfolds if one were to take just a fraction of this maximum maybe even as little as 50%.

Nevertheless, the number of records that can be handled by a 3 and 4 level index is pretty substantial, in particular with larger block sizes.

database block size	Branch Max	Leaf Max	rows/block (in table)	rows/table (in millions)	table size (in GB)
2K	165	122	28	3.30	0.20
4K	331	245	57	26.80	1.90
8K	662	490	114	214.70	15.10

Table 1: Index height = 3

database block size	Branch Max	Leaf Max	rows/block (in table)	rows/table (in millions)	table size (in GB)
2K	165	122	28	548.00	39.10
4K	331	245	57	8,884.80	623.50
8K	662	490	114	142,157.60	9,976.00

Table 2: Index height = 4

The bitmap index

The bitmap indexing technology was introduced in ORACLE7 Release 7.3 and extended in ORACLE8 to address the needs of complex data warehousing queries. It is a technology where for each distinct key there is a bitmap. ORACLE uses B* trees for the physical implementation of bitmap indexes. The bitmaps are broken into pieces in such a way that each piece is not larger than half the size of a data block. These pieces are then stored in the leaf blocks of the B* tree. Each piece has the form

```
<key 1> ... <key n> <start rowid> <end rowid> <bitmap>
```

The start rowid and end rowid specify the range of the table covered by the bitmap. It is this 'range' or 'chunk' that is stored in the B* tree.

Bitmap indexes can be very small in comparison to regular B* tree indexes for low cardinality columns. I.e. when the number of distinct values of those columns is small in comparison to the total number of records in the table. B* tree indexes on the other hand are most efficient for high-cardinality columns. Another advantage of bitmap indexes is their efficiency and fast retrieval due to the ease of bitmap comparisons. AND and OR operations in a WHERE clause can be quickly resolved by simple boolean operations.

There are also a number of disadvantages associated with bitmap indexes. Because of the way a bitmap index is built in ORACLE, applications that require update operations (such as OLTP applications) are not very well suited for the use of bitmap indexes.

The reverse key index

In a reverse key index the bytes of each column in the index except the rowid are reversed. The order of the columns in the index however is not reversed. The reverse key index can solve a potential pinging problem in a Parallel Server implementation when multiple indexes are trying to access the same leaf nodes of a B* tree index. By reversing the keys of the index, the insertions become distributed across the leaf nodes.

There is also a disadvantage of using a reverse key index. Adjacent keys are no longer stored next to one another and therefore it eliminates the possibility of running an index range scan. The only ability that is left is the fetch-by-key or a full-index scan.

The index organized table

In an index organized table the data for the table is kept in its associated index. In other words, the entire record is found in the leaf nodes of the B* tree index. The index structure is the ONLY area where the data is stored. The records are built in the index structure on the primary key for the table. One of the consequences of this setup is that records do not have rowids.

ORACLE also provides a so-called overflow area. In index organized tables, the B* tree index entries can be potentially very large depending on the size of the record. In such a situation, each leaf node may only store one row or even a row piece thus destroying the density property of the tree. The row overflow area fixes this potential problem. If the size of a record is larger than a specified threshold value, the non-key columns exceeding the threshold value are stored in the overflow area.

Some applications that can benefit from index organized tables are spatial applications, OLAP applications and information retrieval applications.

Partitioned indexes

New in ORACLE8 is the concept of partitions. Partitions exist both in table as well as in index objects. Today's companies run database systems sometimes containing several terabytes of data. One way to meet the challenges of these very large databases is by dividing a single, large table or index into smaller pieces or partitions. Each of the partitions share the same logical structure. Each partition is stored in a separate segment and can have different physical attributes. The rules for partitioned indexes are similar to the rules for partitioned tables with some exceptions for cluster indexes and bitmap indexes. However, partitioned indexes are more complicated because there are various types: local prefixed, local non-prefixed and global prefixed. Oracle does not support global non-prefixed indexes.

Local vs. global: In a local index all index keys refer to records of a single underlying table partition only. Local indexes are equi-partitioned with the underlying table partitions. ORACLE maintains these local indexes automatically as partitions are added, dropped or split. In a global index the index keys refer to records that are stored in more than one underlying table partition. Normally, a global index is not equi-partitioned with the underlying table. Even though there is nothing to prevent a global index to be equi-partitioned, the Oracle optimizer can not take advantage of the equi-partitioning.

Prefixed vs. non-prefixed: A prefixed index is partitioned on a left prefix of the index columns. Otherwise the index is non-prefixed.

Local prefixed index: In this case the partition key of the underlying table partition is exactly the same as the left most column keys of the index. Local prefixed indexes can be unique or non-unique.

Local non-prefixed index: This index type refers to the situation where the local index on the table partition is created on a set of columns that are not the same as the table partitioning key. Thus, a unique local non-prefixed index can only be created if and only if the index key is a subset of the partitioning key.

Global prefixed index: A global index is prefixed if it is partitioned on a left prefix of the index columns. They can be unique or non-unique.

Equi-partitioning a table and its indexes through local indexes has a number of advantages. First of all, when a maintenance operation is performed it only affects a single partition. This partition independence supports the smooth rollout of old data as well as the availability of new data in historical tables that is so important in today's data warehouses. Second, the Oracle optimizer can take advantage of the fact that a local index is equi-partitioned to generate better access plans. Finally, local indexes simplify the task of incomplete recoveries. ORACLE8 supports point-in-time recovery of a table. In order to complete such an operation, the associated indexes must be recovered to the same point in time. This is only possible with a local index.

Global indexes are quite a bit harder to manage than local indexes. The main reason is due to the fact that a change to the underlying partition structure (split, move, drop, truncate, etc) causes the entire global index to be affected resulting in the need for the global index to be rebuilt. Furthermore, when a single underlying table partition is recovered to a single point in time the entire global index must be rebuilt as well.

There are also differences in performance among the various index partition types. If an index is prefixed, the Oracle optimizer can resolve the query through a subset of the index partition. When an index is non-prefixed Oracle must apply a predicate to all index partitions. Even to look up a single key Oracle must access all index partitions. Therefore it is less expensive to scan a prefixed index than it is to scan a non-prefixed index.

On the other hand, non-prefixed indexes are very useful for historical data. In a table containing historical data it is common for an index to be defined on a certain column to support fast data access but to be partitioned on the same columns as the underlying table to support the ease of rolling out old data and rolling in new data.

When deciding how to partition indexes on a table a number of aspects must be taken in consideration. Performance, availability, manageability are all variables that will play an important role in the decision which index partition type to use.

Conclusion

The choice of index type in an ORACLE8 system has become a lot tougher due to the fact there are now many more index types. Choosing the wrong type may result in bad performance and additional maintenance operations. A good understanding of the various index types is essential to be able to make the right decision. ■